JustRL: Scaling a 1.5B LLM with a Simple RL Recipe

Bingxiang He[1], **Zekai Qu**[1], **Zeyuan Liu**[1], **Yinghao Chen**[1], **Yuxin Zuo**[1], **Cheng Qian**[2], **Kaiyan Zhang**[1], **Weize Chen**[1], **Chaojun Xiao**[1], **Ganqu Cui**[3], **Ning Ding**[1†], **Zhiyuan Liu**[1†]

[1]Tsinghua University [2]University of Illinois Urbana-Champaign [3]Shanghai AI Lab

[†]**Corresponding Authors.** hebx24@mails.tsinghua.edu.cn

https://huggingface.co/collections/hbx/justrl

https://github.com/thunlp/JustRL

arXiv:2512.16649v1 [cs.CL] 18 Dec 2025

Abstract | Recent advances in reinforcement learning for large language models have converged on increasing complexity: multi-stage training pipelines, dynamic hyperparameter schedules, and curriculum learning strategies. This raises a fundamental question: **Is this complexity necessary?** We present **JustRL**, a minimal approach using single-stage training with fixed hyperparameters that achieves state-of-the-art performance on two 1.5B reasoning models (54.9% and 64.3% average accuracy across nine mathematical benchmarks) while using 2× less compute than sophisticated approaches. The same hyperparameters transfer across both models without tuning, and training exhibits smooth, monotonic improvement over 4,000+ steps without the collapses or plateaus that typically motivate interventions. Critically, ablations reveal that adding "standard tricks" like explicit length penalties and robust verifiers may degrade performance by collapsing exploration. These results suggest that the field may be adding complexity to solve problems that disappear with a stable, scaled-up baseline. We release our models and code to establish a simple, validated baseline for the community.

"Perfection is achieved, not when there is nothing more to add,
but when there is nothing left to take away."

— Antoine de Saint-Exupéry, Airman's Odyssey

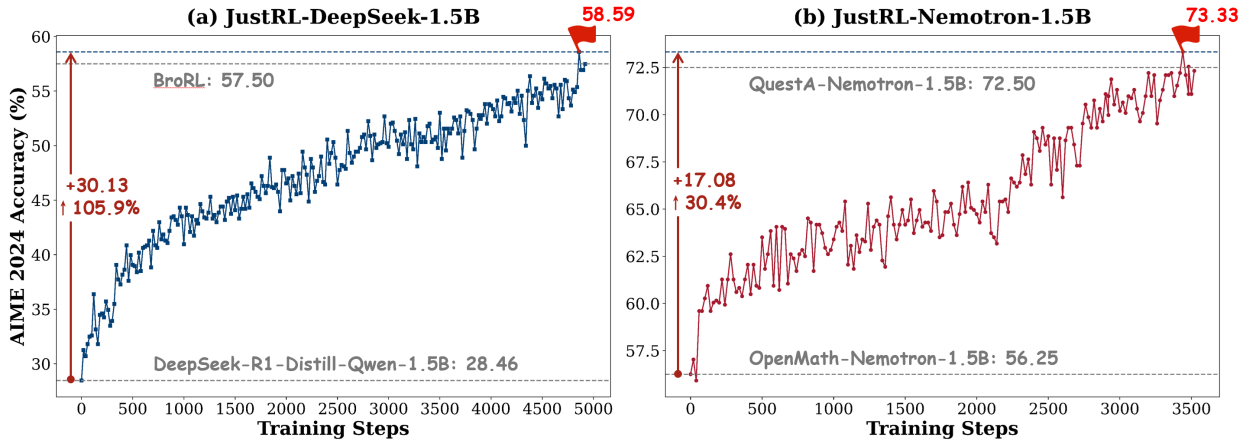

Figure 1 | JustRL achieves substantial performance gains through simple, single-stage training. (a) The AIME24 (avg@32) performance curve for scaling from DeepSeek-R1-Distill-Qwen-1.5B into JustRL-DeepSeek-1.5B, from 28% to 58% over 4,000 steps; (b) from OpenMath-Nemotron-1.5B into our 1.5B reasoning SOTA model JustRL-Nemotron-1.5B, showing its training journey to the final 70+% score over 3,000 steps.

1. Introduction

Recent advances in Large Language Models (LLMs), such as OpenAI's o1 [Jaech et al., 2024] and DeepSeek-R1 [Guo et al., 2025], have demonstrated the remarkable effectiveness of large-scale Reinforcement Learning with Verifiable Rewards (RLVR) for challenging reasoning tasks in mathematics and coding. However, for smaller lightweight models, the field has taken a different path. Leading companies have favored distillation, essentially supervised fine-tuning on outputs from larger teacher models, over direct RL training. This approach makes practical sense: distillation is efficient, stable, and delivers immediate performance gains. Qwen3's strong-to-weak distillation and DeepSeek-R1 both demonstrate the effectiveness of this strategy for small language models (SLMs).

But distillation has a fundamental limitation: it's bounded by the teacher model's capabilities. When researchers rely on distillation to improve the performance of smaller models, they encounter an upper bound, especially when the teacher model's updates are infrequent. Even with increased data and extended training, further gains in performance become difficult to achieve once the teacher model's performance plateaus. In contrast, RL can provide further improvements once the distillation process reaches saturation, making it a crucial approach in such scenarios. Meanwhile, RL for SLMs has gained a reputation for being unstable and difficult, requiring increasingly sophisticated techniques to work reliably. Over the past year, we've seen a proliferation of methods attempting to stabilize and improve RL training for small models: multi-stage training pipelines, dynamic hyperparameter schedules, adaptive temperature controls, response length penalties, and various forms of data curation and filtering [Hu et al., 2025a,b, Li et al., 2025, Liu et al., 2025a, Luo et al., 2025, Min et al., 2024].

This proliferation of techniques raises an important question: **Is this complexity necessary?** When different works combine different subsets of methods and report varying results, it becomes unclear what truly drives performance. More concerning, many recent works cite training instabilities, like reward collapse, entropy drift, and length explosion, as motivation for their techniques, yet apply these techniques on top of already-complex baselines. This makes it impossible to know whether new methods provide genuine benefits or simply compensate for issues introduced by prior complexity. The accumulated "best practices" may be fighting each other rather than the fundamental challenges of RL [Liu et al., 2025d].

In this paper, we explore **whether stable, competitive training can be achieved with a simpler approach.** We apply a minimal setup to two popular 1.5B reasoning models, using single-stage training with fixed hyperparameters derived from common practice. The results match or exceed more complex approaches while using 2× less compute. Importantly, we achieve this without the multi-stage pipelines or dynamic schedules, suggesting that simpler approaches may be sufficient when applied at adequate scale. Besides, the training process itself proves stable: smooth, monotonic improvement over 4,000+ steps without the collapses or oscillations often cited as motivation for complex interventions.

Our goal is not to argue against all techniques or claim we've found the optimal approach. Rather, we provide evidence that simpler baselines deserve more attention than they've received. We offer a simple practice with a minimum set of tricks that can enhance the performance of models that are approaching their distillation limits. The field may benefit from establishing what's fundamentally sufficient before layering on additional complexity.

2. Related Work

Since DeepSeek-R1's release in early 2025, the community has rapidly advanced RL for small language models in mathematical reasoning. The past year has seen a flourishing of approaches, each

Model	EC	THP	TTP	RKL	LC	AT	RR	DS	ST	Date
STILL-3-1.5B	✗	✓	✓	✓	✗	✗	✗	✗	✗	Jan '25
DeepScaleR-1.5B	✓	✗	✗	✗	✓	✗	✗	✗	✓	Feb '25
FastCuRL-1.5B	✗	✓	✗	✗	✓	✗	✗	✗	✓	Mar '25
ProRL-V1	✓	✓	✗	✓	✓	✗	✗	✓	✓	May '25
e3-1.7B	✓	✓	✗	✗	✓	✗	✗	✓	✓	Jun '25
POLARIS-1.7B	✓	✓	✗	✗	✓	✓	✓	✓	✓	Jul '25
ProRL-V2	✓	✓	✗	✓	✓	✗	✗	✓	✓	Aug '25
QuestA-Nemotron	✗	✗	✗	✗	✗	✗	✗	✓	✓	Sep '25
BroRL	✓	✓	✗	✓	✓	✗	✗	✓	✓	Oct '25
JustRL-DeepSeek	✓	✗	✗	✗	✗	✗	✗	✗	✗	Nov '25
JustRL-Nemotron	✓	✗	✗	✗	✗	✗	✗	✗	✗	Nov '25

Table 1 | Comparison of RL techniques used in recent small language models for mathematical reasoning. Model names are colored by backbone: DeepSeek-R1-Distill-Qwen-1.5B, Qwen3-1.7B, OpenMath-Nemotron-1.5B. We use the following abbreviations for RL techniques: EC=Entropy Control, THP=Tune Hyperparameters, TTP=Tune Training Prompt, RKL=Reset KL Reference, LC=Length Control, AT=Adaptive Temperature, RR=Rollout Rescue, DS=Dynamic Sampling, ST=Split Training Stages. Our models (JustRL-DeepSeek and JustRL-Nemotron) use only entropy control, achieving competitive performance with minimal complexity.

introducing techniques to stabilize training and push performance boundaries. These works fall into three main families based on their foundation models, all starting from distilled bases: (1) DeepSeek-R1-Distill-Qwen-1.5B, (2) OpenMath-Nemotron-1.5B, and (3) Qwen3-1.7B.

The evolution reveals a clear trend toward increasing sophistication. Early works like STILL [Min et al., 2024] explored hyperparameter tuning and reference model resets through extensive comparison experiments. Subsequent approaches introduced multi-stage training with progressive context lengthening. DeepScaleR [Luo et al., 2025] divided training into three stages with increasing context lengths (8K → 16K → 24K). FastCuRL [Song et al., 2025] extended this to five stages, alternating between CoT compression (long-to-short) and extension (short-to-long), with each stage using different data, batch sizes, and rollout numbers. ProRL [Liu et al., 2025a] divided training into eight stages with scheduled length penalties, and its successor ProRL-V2 [Hu et al., 2025a] introduced additional techniques including scheduled cosine length penalties while maintaining fixed 8K context. BroRL [Hu et al., 2025b] took a different approach by dramatically increasing rollouts per example to hundreds, aiming to exhaustively explore the solution space after 3K ProRL training steps.

For the OpenMath-Nemotron-1.5B backbone, QuestA [Li et al., 2025] introduced an innovative curriculum learning approach by augmenting questions with partial CoT solutions as hints, providing richer learning signals through staged difficulty progression. On the Qwen3-1.7B backbone, POLARIS [An et al., 2025] employed dynamic dataset filtering to focus on challenging problems, combined with adaptive temperature adjustments and test-time context extrapolation across three

training stages. Similarly, e3 [Setlur et al., 2025] used multi-stage training with varying context lengths and leveraged the model's extrapolation abilities at test time.

Table 1 summarizes these approaches and the techniques they employ. The pattern is striking: nearly every work employs multiple techniques from a growing toolkit, including multi-stage training, adaptive hyperparameters, length penalties, dynamic sampling, and various stabilization mechanisms. While these methods achieve strong results, the accumulated complexity makes it difficult to isolate which elements truly matter. This raises a practical question: Is there a simpler path that still achieves competitive performance?

3. JustRL: Simplicity at Scale

Our approach is deliberately simple. We constrain ourselves to the fundamentals of RL, avoiding the multi-stage pipelines, dynamic schedules, and specialized techniques that have become common in recent work.

3.1. Training Setup

Core algorithm. We use default implementation of GRPO in veRL [Sheng et al., 2025] with binary outcome rewards. The reward signal comes from a lightweight rule-based verifier from DAPO [Yu et al., 2025], without symbolic math libraries like SymPy that could add computational overhead.

What we keep simple:

- **Single-stage training:** No progressive context lengthening, no curriculum switching, no stage transitions. We train continuously from start to finish.
- **Fixed hyperparameters:** No adaptive temperature scheduling, no dynamic batch size adjustments, no mid-training reference model resets.
- **Standard data:** We train on DAPO-Math-17k [Yu et al., 2025] without offline difficulty filtering or online dynamic sampling strategies.
- **Basic prompting:** A simple suffix prompt without tuning: "Please reason step by step, and put your final answer within \boxed{}."
- **Length control:** We simply set the maximum context length as 16K tokens, rather than using explicit length penalty terms.

The one technique we do use. We employ "clip higher", a well-established practice for stability in long-horizon RL training. We view this as part of the baseline rather than an added technique.

We train this recipe on two 1.5B reasoning models using veRL: DeepSeek-R1-Distill-Qwen-1.5B and OpenMath-Nemotron-1.5B, each with 32 A800-80GB GPUs for ~15 days. The same hyperparameters work for both, without per-model tuning, and remain fixed throughout training. Table 2 shows the complete hyperparameter configuration.

3.2. Evaluation Protocol

We evaluate nine challenging mathematical reasoning tasks based on reproducible evaluation scripts from POLARIS [An et al., 2025]:

- **Benchmarks:** AIME 2024 [Li et al., 2024], AIME 2025 [Balunović et al., 2025], AMC 2023 [Li et al., 2024], MATH-500 [Hendrycks et al., 2021], Minerva Math [Lewkowycz et al., 2022], OlympiadBench [He et al., 2024], HMMT Feb 2025 [Balunović et al., 2025], CMIMC 2025 [Balunović et al., 2025] and BRUMO 2025 [Balunović et al., 2025].

Hyperparameter	Value
Advantage Estimator	GRPO
Use KL Loss	No
Use Entropy Regularization	No
Train Batch Size	256
Max Prompt Length	1k
Max Response Length	15k
PPO Mini Batch Size	64
PPO Micro Batch Size/GPU	1
Clip Ratio Range	[0.8, 1.28]
Learning Rate	1e-6 (constant)
Temperature	1.0
Rollout N	8
Reward Function	DAPO [Yu et al., 2025]

Table 2 | Fixed hyperparameter configuration used for both JustRL models.

- **Evaluation protocol:** We report Pass@1 accuracy, averaging over N sampled responses per problem (N=4 for MATH-500, Minerva Math, and OlympiadBench; N=32 for others). We use temperature 0.7, top-p 0.9, and allow up to 32K tokens for generation.

We augment existing systems with CompassVerifier-3B [Liu et al., 2025c], a lightweight model-based verifier, to address false negatives from rule-based verifiers.

4. Experimental Results

We apply JustRL on two popular 1.5B reasoning models to demonstrate that our minimal recipe achieves competitive performance with notably stable training dynamics.

4.1. Scaling a Weaker Base: JustRL-DeepSeek-1.5B

Takeaway 1

Starting from DeepSeek-R1-Distill-Qwen-1.5B, we achieve better results through single-stage training with fixed hyperparameters, outperforming more complex approaches while using 2× less compute. The training curve shows over 4,000 steps of stable improvement without intervention, suggesting that an adequate scale with simple methods can outperform sophisticated techniques.

We train DeepSeek-R1-Distill-Qwen-1.5B for 4,380 steps using our simple, single-stage recipe. We report the avg@32 results across nine mathematical benchmarks in Table 3.

Results. Our model (JustRL-DeepSeek-1.5B) achieves 54.87% average across benchmarks, outperforming ProRL-V2's 53.08% despite ProRL-V2's nine-stage training pipeline with dynamic hyperparameters and more sophisticated techniques. We also lead on six of nine benchmarks, demonstrating broad improvements rather than overfitting to a single task.

Computational efficiency. However, the real question is whether our simplicity comes at a computational cost. It doesn't. Table 4 compares the computational cost across methods. We match half of ProRL-V2's compute budget while using a single-stage recipe with fixed hyperparameters. BroRL requires 4.9× more compute by increasing rollouts to 512 per example, essentially exhaus-

Model	AIME24	AIME25	AMC23	MATH	Minerva	Olympiad	HMMT	BRUMO	CMIMC	Avg
Backbone	29.90	22.40	63.82	84.90	34.65	45.95	13.44	30.94	12.89	37.65
DeepScaleR-1.5B	40.21	28.65	73.83	89.30	39.34	52.79	18.96	40.00	21.00	44.88
ProRL-V2	51.87	35.73	88.75	92.00	49.03	67.84	19.38	47.29	**25.86**	53.08
BroRL*	**57.50**	36.88	–	**92.14**	49.08	61.54	–	–	–	–
JustRL-DeepSeek	52.60	**38.75**	**91.02**	91.65	**51.47**	**67.99**	**21.98**	**52.71**	25.63	**54.87**

Table 3 | Results on DeepSeek-R1-Distill-Qwen-1.5B backbone. All scores except MATH-500, Minerva, and OlympiadBench use @32 sampling; those three use @4. *BroRL results are officially reported but models not released; some benchmarks unavailable.

Model	Dynamic Sampling*	Training Steps	Train Batch Size	Rollout N	Max Context Length	Token Budget (approx.)
DeepScaleR-1.5B	✗	1,750	128	8	8k→16k→24k	2.2×10^{6}k
ProRL-V1	✓	2,450	256	16→32→16	8k→16k	2.1×10^{8}k
ProRL-V2	✓	+1,000	256	16→32→16	8k→16k→8k	2.8×10^{8}k
BroRL	✓	+191	128	512	16k	6.8×10^{8}k
JustRL-DeepSeek	✗	4,380	256	8	16k	1.4×10^{8}k

Table 4 | Computational cost comparison for DeepSeek-R1-Distill-Qwen-1.5B based models. *Dynamic sampling with estimated 50% filter ratio following POLARIS [An et al., 2025]. ProRL-V2 continues from ProRL-V1 (+1,000 steps), and BroRL continues from ProRL-V2 (+191 steps).

tively exploring the solution space. Our approach achieves competitive performance without this computational overhead.

Note on dynamic sampling. Models marked with ✓ use dynamic sampling to filter examples. Following POLARIS [An et al., 2025], we estimate a 50% filter ratio for DeepSeek-R1-Distill-Qwen-1.5B using dynamic sampling, as rollouts often contain many trivial/hard cases (e.g., 8/8 or 0/8 correct rollouts). Even assuming no filtering (i.e., 0% ratio), our compute use remains comparable or even lower, making our estimates conservative.

Training stability. Figure 1(a) shows our training curve for JustRL-DeepSeek-1.5B, showing smooth and monotonic improvement without the oscillations or plateaus that typically require intervention. The stability itself suggests we're not fighting against our training setup.

4.2. Scaling a Stronger Base: JustRL-Nemotron-1.5B

Takeaway 2

The same recipe scales OpenMath-Nemotron-1.5B to the current best math reasoning performance without any hyperparameter adjustment, matching state-of-the-art results that use curriculum learning and question augmentation. Competitive performance across two different starting points suggests the approach is robust rather than carefully tuned to specific conditions.

We train OpenMath-Nemotron-1.5B for 3,440 steps using the identical recipe, without hyperparameter changes. We report the evaluation results across nine challenging mathematical benchmarks in Table 5.

Results. We achieve 64.32% average, slightly outperforming QuestA's 63.81% and leading on five of

Model	AIME24	AIME25	AMC23	MATH	Minerva	Olympiad	HMMT	BRUMO	CMIMC	Avg
Backbone	58.75	48.44	90.55	92.40	26.93	71.70	30.10	61.67	30.08	56.74
QuestA	**71.56**	62.08	93.44	92.95	**32.08**	72.28	**40.94**	**67.50**	41.48	63.81
JustRL-Nemotron	69.69	**62.92**	**96.02**	**94.15**	30.24	**76.59**	40.63	66.88	**41.72**	**64.32**

Table 5 | Results on OpenMath-Nemotron-1.5B backbone. All scores except MATH-500, Minerva, and OlympiadBench use @32 sampling; those three use @4.

Model	Dynamic Sampling*	Training Steps	Train Batch Size	Rollout N	Max Context Length	Token Budget (approx.)
QuestA	✔	2,000	128	16	32k	2.6×10^{8}k
JustRL-Nemotron	✗	3,440	256	8	16k	1.1×10^{8}k

Table 6 | Computational cost comparison for OpenMath-Nemotron-1.5B based models. *Dynamic sampling with estimated 50% filter ratio. Despite more training steps, JustRL-Nemotron uses 2.4× less compute.

nine benchmarks. The gap is narrow, which makes sense. Both approaches are pushing the boundaries of what's achievable at 1.5B scale. The key difference is in how we get there.

QuestA introduces an innovative curriculum learning approach that augments questions with partial CoT solutions as hints, splitting training stages with different difficulty. This requires not just ground-truth answers but full reasoning trajectories generated by larger models for curriculum construction with additional data requirements and engineering complexity. Our approach uses only the standard question-answer pairs without augmentation or curriculum design.

Computational efficiency. We use 2× less compute while achieving slightly better average performance without designing a complex curriculum as used in QuestA.

Training stability. Figure 1(b) shows another smooth training curve. The fact that the same recipe works for both models without hyperparameter tuning suggests genuine robustness rather than lucky optimization for a single model.

These results don't diminish QuestA's contribution, where question augmentation is a clever technique that clearly helps. Rather, they demonstrate that competitive performance is achievable through simpler means.

4.3. Training Dynamics Analysis

The ultimate test of a training recipe isn't just the final numbers; it's whether you can get there reliably. Complex techniques often emerge as responses to training instability: oscillating rewards, collapsing policies, or runaway response lengths. If a simpler approach can avoid these failure modes entirely, it suggests we may have been treating symptoms rather than causes. We examine the training dynamics of JustRL-DeepSeek-1.5B in detail, tracking three key dynamics over 4,000 training steps: mean training reward, policy entropy, and mean response length in Figure 2. These dynamics reveal whether the model is learning stably or requires constant intervention.

- **Entropy:** Figure 2(a) shows policy entropy oscillating between 1.0 and 1.6 at later training steps, with no systematic drift upward (exploration collapse) or downward (premature convergence), indicating that the simple "clip higher" technique is well-performed for large-scale RL.

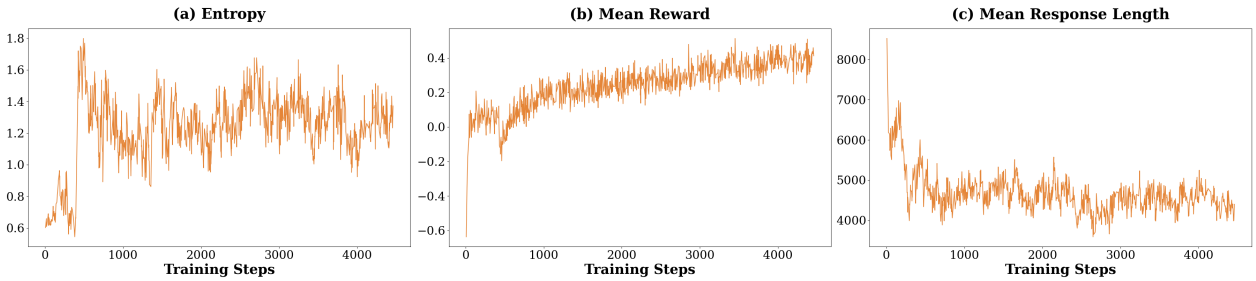

Figure 2 | Training Dynamics of JustRL-DeepSeek-1.5B. (a) Policy entropy remains stable throughout training, oscillating naturally around 1.2-1.4 without drift or collapse. (b) Mean reward shows smooth, monotonic improvement from negative to ~0.4, indicating consistent learning without plateau-breaking interventions. (c) Response length naturally converges from initial verbosity (~7,000 tokens) to a stable range (4,000-5,000 tokens) with 16k max context length, without explicit length penalties.

- **Mean Reward:** Figure 2(b) shows the mean reward climbing from around -0.6 to +0.4 over training. The curve is noisy but the trend is unmistakably upward. More importantly, there are no extended plateaus or sudden drops that would typically trigger intervention in multi-stage approaches. The signal is consistent enough that the model can learn continuously.
- **Mean Response Length:** The model starts verbose, generating responses averaging ~8,000 tokens. Without any explicit length penalty, it naturally compresses to 4,000-5,000 tokens by step 1,000 and maintains this range. This organic compression may be more robust than explicit penalties, which can create adversarial pressure that models learn to game [Liu et al., 2025b].

The contrast with typical RL: While we don't have the computational resources to run extensive controlled comparisons, the literature provides context. Many recent works explicitly cite training instabilities as motivation for their techniques: ProRL-v2 [Hu et al., 2025a] introduces scheduled length penalties after observing length drift; BroRL [Hu et al., 2025b] increases rollouts to hundreds after hitting plateaus; multiple works [Liu et al., 2025a, Min et al., 2024] apply KL regularization and reset reference models when KL divergence grows too large, which limits the training upper bound. Our training exhibits none of these pathologies that motivate intervention.

What we can't claim: These smooth curves don't prove that simpler approaches are always more stable, or that techniques never help. We can't isolate which specific complex techniques cause instability versus which ones solve it. But the contrast is striking: a minimal recipe produces training dynamics that simply don't require the interventions that have become standard practice.

4.4. Ablation Studies

We conduct two ablation studies starting from our base recipe on JustRL-DeepSeek-1.5B, both trained for 3,000+ steps:

- **w/ Overlong Penalty:** Add an explicit length penalty term for the last 4k tokens (as used in DAPO [Yu et al., 2025])
- **w/ Overlong Penalty + Robust Verifier:** Further add a more sophisticated verifier from DeepScaleR [Luo et al., 2025] to reduce false negatives

Results. Figure 3 shows that both modifications degrade performance: adding overlong penalty plateaus at 50% AIME 2024 (vs 55% baseline), and adding both modifications plateaus at 45%.

On the overlong penalty. We hypothesized that explicitly penalizing verbose responses might improve

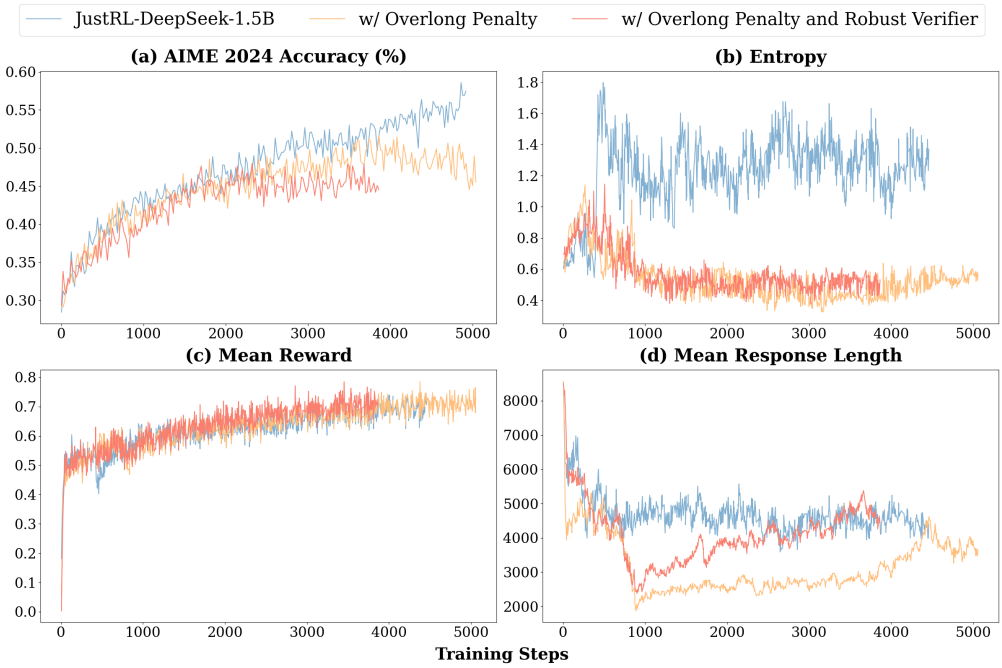

Figure 3 | Ablation Study Results. (a) AIME 2024 performance diverges after ~2,000 steps. Our base recipe reaches 55%, while adding overlong penalty plateaus at 50%, and adding both modifications plateaus at 45%. (b) Entropy: Both modifications show collapsed exploration (entropy ~0.5-0.6) compared to healthy oscillation in the base recipe (~1.2-1.4).

training efficiency by pushing the model toward conciseness faster. Instead, performance degraded significantly as a trade-off. The entropy plot in Figure 3(b) reveals why: the explicit penalty collapses exploration, driving entropy down to 0.5-0.6 compared to the 1.2-1.4 range in our base approach. The explicit penalty appears to create pressure that conflicts with the learning objective, forcing premature convergence to shorter responses before the model has explored what actually works.

On the robust verifier. We further hypothesized that reducing false negatives (correct solutions marked wrong) would provide a cleaner learning signal. However, even after normalizing reward scales, its use leads to worse final performance, plateauing at 45% AIME 2024. We offer two possible explanations: first, the stricter base verifier creates a richer spectrum of learning signals by reducing "perfect" scores, whereas the robust verifier's permissiveness offers less nuanced guidance. Second, the stricter verifier's reliance on precise formatting may pressure the model to develop more robust internal computations, an incentive lost when the verifier corrects errors externally. Thus, a forgiving verifier might fail to encourage the precision required for optimal generalization.

These results reveal two important lessons. First, not all "standard tricks" transfer across contexts. The overlong penalty works in DAPO's setting [Yu et al., 2025] but degrades performance in ours, demonstrating that techniques interact with other design choices in complex and sometimes unpredictable ways. Second, simpler approaches are not always easier to improve. We tested two seemingly reasonable modifications and both made things worse, suggesting our base recipe achieves a delicate balance that is easily disrupted by additional interventions.

We want to be clear about the limits of these ablations. We tested two specific modifications, but many other techniques remain unexplored: curriculum learning, adaptive temperature scheduling, reference model resets, different verifier designs, and various forms of data augmentation. Some of these might improve upon our baseline. Our point is not that additional techniques *never* help, rather,

it is that they should be validated empirically rather than assumed to be beneficial.

5. Discussion

What this suggests: The smooth training curves with healthy entropy, monotonic rewards and natural length convergence stand in contrast to instabilities often cited as motivation for complex techniques. Our negative ablations show that adding "improvements" actively degrades performance. This suggests complexity may sometimes address symptoms created by other design choices rather than fundamental RL challenges.

What we don't know: We demonstrate that simple RL works well, but can't isolate why. Is it the hyperparameters? The training dataset? The verifier design? The interaction between all three? Our results are also limited to two backbones in mathematical reasoning at 1.5B scale. Generalization to other domains, model sizes, and tasks remains an open question.

When might complexity help: We don't advocate simplicity as dogma. Additional techniques may be valuable under extreme compute constraints, when encountering specific failure modes we didn't face, when pushing beyond current performance ceilings, or in domains with noisier reward signals. Our argument is methodological: **establish simple baselines first, then add complexity only when you identify specific problems it solves.**

6. Conclusion

The debate over RL for small models has been clouded by assumptions that complexity is necessary for stability and performance. We set out to answer a straightforward question: What happens if we apply RL to small language models without specialized techniques that have become standard practice? By stepping back to a simpler approach, our findings provide a clear answer: adequate scale with stable fundamentals can match sophisticated techniques. Starting from two foundation models, we achieved comparable or better performance using single-stage training with fixed hyperparameters, matching or exceeding approaches that employ multi-stage training and curriculum learning while using 2× less compute. More striking than the final numbers is the path: smooth, stable improvement over thousands of steps without the interventions typically required to prevent training collapse. We advocate a methodological shift: **start simple, scale up, and only add complexity when a simple, robust baseline demonstrably fails.** If simplicity is sufficient more often than current practice assumes, that seems worth paying attention to.

Limitations

Our work has several limitations. First, our results are limited to mathematical reasoning tasks at the 1.5B parameter scale, and generalization to other domains (e.g., coding, general question answering) and model sizes remains unexplored. Second, while we demonstrate that simplicity works, we cannot definitively isolate which specific components (hyperparameters, verifier design, training data) are most critical to our success. Third, our compute budget, while lower than some complex methods, may still be prohibitive for resource-constrained researchers. Finally, we have not explored whether our approach maintains advantages when pushed to even longer training horizons or whether additional techniques might become necessary at scale.

References

Chenxin An, Zhihui Xie, Xiaonan Li, Lei Li, Jun Zhang, Shansan Gong, Ming Zhong, Jingjing Xu, Xipeng Qiu, Mingxuan Wang, and Lingpeng Kong. Polaris: A post-training recipe for scaling reinforcement learning on advanced reasoning models, 2025. URL https://hkunlp.github.io/blog/2025/Polaris.

Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. Matharena: Evaluating llms on uncontaminated math competitions. *arXiv preprint arXiv:2505.23281*, 2025.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025.

Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. OlympiadBench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3828–3850, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.211. URL https://aclanthology.org/2024.acl-long.211/.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

Jian Hu, Mingjie Liu, Shizhe Diao, Ximing Lu, Xin Dong, Pavlo Molchanov, Yejin Choi, Jan Kautz, and Yi Dong. Prorl v2: Prolonged training validates rl scaling laws, August 2025a. URL https://hijkzzz.notion.site/prorl-v2. First published on Notion.

Jian Hu, Mingjie Liu, Ximing Lu, Fang Wu, Zaid Harchaoui, Shizhe Diao, Yejin Choi, Pavlo Molchanov, Jun Yang, Jan Kautz, et al. Brorl: Scaling reinforcement learning via broadened exploration. *arXiv preprint arXiv:2510.01180*, 2025b.

Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.

Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al. Solving quantitative reasoning problems with language models. *Advances in Neural Information Processing Systems*, 35: 3843–3857, 2022.

Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Huang, Kashif Rasul, Longhui Yu, Albert Q Jiang, Ziju Shen, et al. Numinamath: The largest public dataset in ai4maths with 860k pairs of competition math problems and solutions. *Hugging Face repository*, 13:9, 2024.

Jiazheng Li, Hongzhou Lin, Hong Lu, Kaiyue Wen, Zaiwen Yang, Jiaxuan Gao, Yi Wu, and Jingzhao Zhang. Questa: Expanding reasoning capacity in llms via question augmentation. *arXiv preprint arXiv:2507.13266*, 2025.

Mingjie Liu, Shizhe Diao, Ximing Lu, Jian Hu, Xin Dong, Yejin Choi, Jan Kautz, and Yi Dong. Prorl: Prolonged reinforcement learning expands reasoning boundaries in large language models. *arXiv preprint arXiv:2505.24864*, 2025a.

Shih-Yang Liu, Xin Dong, Ximing Lu, Shizhe Diao, Mingjie Liu, Min-Hung Chen, Hongxu Yin, Yu-Chiang Frank Wang, Kwang-Ting Cheng, Yejin Choi, et al. Dler: Doing length penalty right-incentivizing more intelligence per token via reinforcement learning. *arXiv preprint arXiv:2510.15110*, 2025b.

Shudong Liu, Hongwei Liu, Junnan Liu, Linchen Xiao, Songyang Gao, Chengqi Lyu, Yuzhe Gu, Wenwei Zhang, Derek F. Wong, Songyang Zhang, and Kai Chen. Compassverifier: A unified and robust verifier for large language models. 2025c.

Zihe Liu, Jiashun Liu, Yancheng He, Weixun Wang, Jiaheng Liu, Ling Pan, Xinyu Hu, Shaopan Xiong, Ju Huang, Jian Hu, et al. Part i: Tricks or traps? a deep dive into rl for llm reasoning. *arXiv preprint arXiv:2508.08221*, 2025d.

Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Li Erran Li, Raluca Ada Popa, and Ion Stoica. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl. https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a4e2, 2025. Notion Blog.

Yingqian Min, Zhipeng Chen, Jinhao Jiang, Jie Chen, Jia Deng, Yiwen Hu, Yiru Tang, Jiapeng Wang, Xiaoxue Cheng, Huatong Song, et al. Imitate, explore, and self-improve: A reproduction report on slow-thinking reasoning systems. *arXiv preprint arXiv:2412.09413*, 2024.

Amrith Setlur, Matthew YR Yang, Charlie Snell, Jeremy Greer, Ian Wu, Virginia Smith, Max Simchowitz, and Aviral Kumar. e3: Learning to explore enables extrapolation of test-time compute for llms. *arXiv preprint arXiv:2506.09026*, 2025.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297, 2025.

Mingyang Song, Mao Zheng, Zheng Li, Wenjie Yang, Xuan Luo, Yue Pan, and Feng Zhang. Fastcurl: Curriculum reinforcement learning with stage-wise context scaling for efficient training r1-like reasoning models. *arXiv preprint arXiv:2503.17287*, 2025.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.